

A QUICK REFERENCE MANUAL

Git & GitHub Cookbook

Version Control · Collaboration



Luigi Giuffrida

Department of Electronics and Telecommunications
Politecnico di Torino

Draft — September 11, 2026

This booklet is a companion reference, not a full course. It covers the Git commands and GitHub workflow that students need on a day-to-day basis, with just enough theory to understand what the commands actually do. For anything beyond this scope, the official documentation at git-scm.com/doc is the authoritative source.

Contents

1	What Git Actually Does	1
1.1	Version control, in one sentence	1
1.2	Three places your files can be	1
1.3	Commits, hashes and branches	2
1.4	Local versus remote	2
1.5	Checklist	2
2	Everyday Commands	3
2.1	One-time setup	3
2.2	Starting a repository	3
2.3	The basic loop: status, add, commit	3
2.4	Inspecting history	4
2.5	Branching and merging	4
2.6	Undoing things	5
2.7	<code>.gitignore</code>	5
2.8	Checklist	5
3	Working with GitHub	7
3.1	Remotes	7
3.2	Forking versus branching	7
3.3	Pull requests	8
3.4	Issues and project boards	8
3.5	A minimal README	8
3.6	Checklist	9
A	One-Page Quick Reference	11

What Git Actually Does

The minimum theory needed before the commands make sense: what a repository is, what a commit is, and where your files actually live at each stage.

1.1 Version control, in one sentence

Git is a tool that records snapshots of a set of files over time, so that you can go back to any earlier snapshot, see what changed between two snapshots, and combine changes made by different people. Each snapshot is called a **commit**. Unlike copying a folder by hand, every commit is linked to the one before it, forming a history you can inspect, search and revert.

Git is **distributed**: every clone of a repository is a full copy of the entire history, not just a checkout of the current state. There is no central server that must be reachable to commit, browse history, or create a branch; a server is only needed to synchronise with other people.

1.2 Three places your files can be

Understanding Git means understanding that a tracked file exists in up to three places at once.

1. **Working directory.** The files you see and edit on disk.
2. **Staging area** (also called the *index*). A holding area where you assemble exactly the changes that will go into the next commit. This is what makes Git different from most other tools: you choose what to commit, not everything you touched.
3. **Repository.** The database of committed snapshots, stored in the hidden `.git` folder. Once a change is committed, it is part of the permanent history.

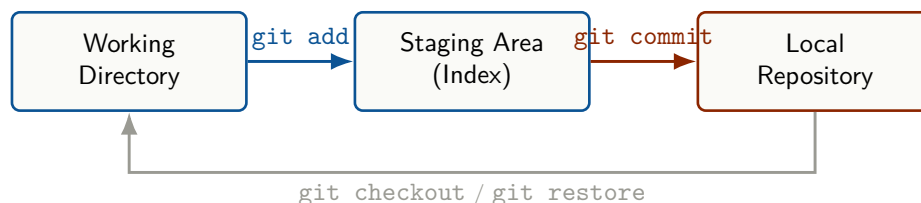


Figure 1.1. The everyday local workflow: edit, stage, commit.

△ Trap: "I saved the file, so it's in Git"

Saving a file only changes the working directory. It is not staged until you run `git add`, and it is not part of the history until you run `git commit`. Three separate, deliberate steps.

1.3 Commits, hashes and branches

A commit is a snapshot of the whole project (not a diff), together with an author, a timestamp, a message, and a pointer to its parent commit. Git identifies each commit by a SHA-1 hash (e.g. `a3f1c9e...`), computed from its content: change one byte anywhere in the project and every commit made afterwards gets a new hash.

A **branch** is nothing more than a movable pointer to a commit. The default branch is conventionally called `main` (older repositories use `master`). Creating a branch is instantaneous and cheap because it does not copy any files, only records a new pointer. `HEAD` is a pointer to whichever branch (or commit) you currently have checked out.

If you remember one thing from this chapter: a commit is a snapshot, a branch is a label on a commit, and `HEAD` says which label you are standing on.

1.4 Local versus remote

Everything so far happens on your machine. A **remote** is simply another copy of the repository, usually hosted on a server such as GitHub, identified by a short name (conventionally `origin`) and a URL. `git push` sends your local commits to a remote branch; `git fetch` downloads commits from a remote without touching your working directory; `git pull` does a fetch followed by a merge (or rebase) into your current branch. Chapter 3 covers this in detail.

1.5 Checklist

- A file only enters history through **add** then **commit**; there is no implicit save.
- A commit is a full snapshot identified by a content hash, with a pointer to its parent.
- A branch is a pointer, not a copy; `HEAD` shows which one you are on.
- Every clone holds the entire history; a remote is just another copy to synchronise with.

Everyday Commands

The commands you will type dozens of times a day, grouped by task, with the flags that actually matter. Everything here is meant to be looked up, not memorised in one sitting.

2.1 One-time setup

Run once per machine, so every commit is correctly attributed.

```
git config --global user.name "Luigi Giuffrida"
git config --global user.email "luigi.giuffrida@polito.it"
git config --global init.defaultBranch main
git config --global core.editor "nano" # or vim, code --wait, ...
```

2.2 Starting a repository

Command	What it does
<code>git init</code>	Turns the current folder into a new, empty repository.
<code>git clone <url></code>	Copies an existing remote repository, including its full history, into a new local folder.
<code>git clone <url> <dir></code>	Same, but names the local folder <code><dir></code> instead of using the remote's default name.

2.3 The basic loop: status, add, commit

```
git status          # what changed, what is staged
git diff            # unstaged changes, line by line
git diff --staged   # staged changes, line by line
git add file1.sv file2.sv # stage specific files
git add .           # stage everything in and below the current dir
git commit -m "Fix reset polarity in cordic_rot"
git commit -am "message" # stage all tracked, modified files, then commit
```

Recipe: Write commit messages that will help future-you

Use the imperative mood ("Fix", not "Fixed" or "Fixes") and keep the first line under about 50 characters. If you need to explain why, leave a blank line and write a longer body below. `git log` then shows a clean one-line summary per commit.

△ Trap: `git add .` adds everything, including junk

Untracked build artefacts, editor swap files and simulator logs get staged along with real changes. Add a `.gitignore` before your first commit (Section 2.7), and check `git status` before every commit.

2.4 Inspecting history

Command	What it does
<code>git log</code>	Full commit history, newest first.
<code>git log -oneline -graph -all</code>	Compact view with the branch topology; the single most useful log incantation.
<code>git show <commit></code>	The full diff introduced by one commit.
<code>git blame <file></code>	Which commit last touched each line.
<code>git diff <a>..</code>	Everything that changed between two commits or branches.

2.5 Branching and merging

```
git branch                # list local branches
git branch feature/uart   # create a branch (does not switch to it)
git switch feature/uart   # switch to it
git switch -c feature/uart # create and switch in one step
git switch main
git merge feature/uart     # bring feature/uart's commits into main
git branch -d feature/uart # delete, once merged
```

switch/restore vs. checkout

`git checkout` used to do both "switch branch" and "discard a file's changes", which was a frequent source of accidents. Modern Git (2.23+) splits these into `git switch` (branches) and `git restore` (files). Both forms work; prefer the new ones.

Merge conflicts

A conflict happens when the same lines were changed differently on both branches. Git stops the merge and marks the file:

```
<<<<<< HEAD
q <= d;
=====
q <= d_reg;
>>>>>> feature/uart
```

Edit the file to the version you want, remove the marker lines, then:

```
git add conflicted_file.sv
git commit    # completes the merge
```

2.6 Undoing things

Pick the row that matches how far the change has gone.

Situation	Command
Discard unstaged edits to a file	<code>git restore <file></code>
Unstage a file, keep the edits	<code>git restore -staged <file></code>
Fix the message of the last commit	<code>git commit -amend</code>
Undo the last commit, keep the changes staged	<code>git reset -soft HEAD~1</code>
Undo the last commit, discard the changes	<code>git reset -hard HEAD~1</code>
Undo a commit that is already pushed	<code>git revert <commit></code>
Set aside unfinished work temporarily	<code>git stash / git stash pop</code>

△ **Trap:** `reset -hard` deletes work, permanently

`git reset -hard` rewrites the working directory to match the target commit and discards uncommitted changes with no confirmation. `git revert` is the safe alternative once a commit has been shared with anyone else: it adds a new commit that undoes the old one, instead of rewriting history.

2.7 .gitignore

A plain text file, one pattern per line, listing what Git should never track: build products, logs, editor files, credentials.

```
# simulation and build output
*.log
*.vcd
work/
obj_dir/

# editor and OS clutter
.vscode/
.DS_Store
```

Patterns are read from every `.gitignore` in the tree; a leading `/` anchors a pattern to that folder, and a trailing `/` matches directories only. A file already tracked before it was added to `.gitignore` keeps being tracked; remove it once with `git rm -cached <file>`.

2.8 Checklist

- `status` and `diff` before every `add`; know what you are about to commit.
- Small, focused commits with an imperative-mood message beat one giant commit at the end of the day.
- `switch/restore` for everyday work; save `checkout` for detached-HEAD or older scripts.
- `revert` once a commit is shared; `reset -hard` only on commits nobody else has seen.
- Add a `.gitignore` before the first commit, not after.

Working with GitHub

GitHub hosts Git repositories and adds collaboration on top: pull requests, issues, and access control. This chapter covers the workflow you need to contribute to a shared repository.

3.1 Remotes

A remote is a named reference to another copy of the repository. Cloning a repository creates one automatically, named `origin`.

```
git remote -v                # list remotes and their URLs
git remote add origin git@github.com:user/repo.git
git push -u origin main      # push, and remember this as the default upstream
git push                     # afterwards, this alone is enough
git fetch origin             # download commits, do not touch the working dir
git pull                     # fetch, then merge into the current branch
```

Recipe: SSH over HTTPS

An HTTPS remote (`https://github.com/...`) asks for a username and a personal access token on every push. An SSH remote (`git@github.com:...`) uses a key pair once configured (`ssh-keygen`, then add the public key under GitHub → Settings → SSH and GPG keys) and never prompts again. Prefer SSH for any repository you push to regularly.

3.2 Forking versus branching

Two different ways to propose a change, depending on your access level.

You have write access	You do not
Create a branch directly in the shared repository.	Fork the repository into your own account (a full server-side copy), clone your fork, and work there.

```
# after forking user/repo into your-name/repo on github.com
git clone git@github.com:your-name/repo.git
cd repo
git remote add upstream git@github.com:user/repo.git  # the original
git fetch upstream
git merge upstream/main          # keep your fork up to date
```

3.3 Pull requests

A **pull request** (PR) asks the maintainers of a repository to merge a branch (yours, or one in your fork) into theirs. It is where code review happens: comments, requested changes, and automated checks all attach to the PR, not to individual commits.

1. Create a branch with a descriptive name: `git switch -c fix/uart-parity`.
2. Commit your changes, then `git push -u origin fix/uart-parity`.
3. On GitHub, open a pull request from that branch into the target branch (usually `main`).
4. Address review comments by committing more changes to the same branch and pushing again; the PR updates automatically.
5. Once approved, merge it (or let a maintainer merge it) and delete the branch.

One pull request, one concern. A PR that fixes a bug, reformats unrelated code and adds a new feature is nearly impossible to review properly.

△ Trap: Diverged history after a rewritten branch

If a branch's history was rewritten upstream (a rebase, an `-amend` that was already pushed), a plain `git pull` will refuse or create a tangled merge. `git pull --rebase` replays your local commits on top of the new history instead; ask before force-pushing a shared branch, since it can overwrite other people's work.

3.4 Issues and project boards

Issues track bugs, tasks and questions independently of any particular commit. A commit message or PR description containing `Fixes #12` automatically closes issue 12 when that commit lands on the default branch. Labels, milestones and project boards organise issues but are GitHub features layered on top of Git itself, not part of the repository history.

3.5 A minimal README

Every repository should have a top-level `README.md`, since GitHub renders it automatically on the repository's front page.

```
# Project name

One paragraph: what this is and why it exists.

## Build
make all

## Layout
src/      source files
test/     testbenches / unit tests
```

3.6 Checklist

- `origin` is the remote you cloned from; add `upstream` when working from a fork.
- Use SSH remotes for repositories you push to often.
- One branch, one pull request, one concern.
- Never force-push a branch other people are also working on without asking first.
- A short `README.md` and a `.gitignore` are the two files every repository should start with.

One-Page Quick Reference

Everything in this booklet, compressed onto one page to keep next to the keyboard.

Command	Purpose
<code>git init</code>	Create a new repository here
<code>git clone <url></code>	Copy a remote repository, with history
<code>git status</code>	What changed, what is staged
<code>git diff / -staged</code>	Line-by-line unstaged / staged changes
<code>git add <file> / .</code>	Stage a file / everything
<code>git commit -m "..."</code>	Record a snapshot of the staged changes
<code>git commit -amend</code>	Rewrite the last commit
<code>git log -oneline -graph -all</code>	Compact history with branches
<code>git show <commit></code>	Full diff of one commit
<code>git blame <file></code>	Who last changed each line
<code>git branch</code>	List local branches
<code>git switch -c <name></code>	Create and check out a branch
<code>git switch <name></code>	Check out an existing branch
<code>git merge <branch></code>	Merge a branch into the current one
<code>git restore <file></code>	Discard unstaged changes to a file
<code>git restore -staged <file></code>	Unstage a file, keep the edits
<code>git reset -soft HEAD~1</code>	Undo last commit, keep changes staged
<code>git reset -hard HEAD~1</code>	Undo last commit, discard changes
<code>git revert <commit></code>	Add a commit that undoes another (safe, shared history)
<code>git stash / stash pop</code>	Set aside / restore unfinished work
<code>git remote -v</code>	List remotes
<code>git push -u origin <branch></code>	Push and set the default upstream
<code>git pull / pull -rebase</code>	Fetch and merge / replay onto new history
<code>git fetch</code>	Download commits without touching the working dir

The five traps to remember

1. Editing a file does not stage it, and staging does not commit it – three separate steps.
2. `git add .` stages everything, junk included; keep a `.gitignore` from the first commit.
3. `reset -hard` discards work with no confirmation; use `revert` on anything already pushed.
4. Never force-push a branch someone else is also working on.
5. One pull request, one concern.